

Introduction to

Programming with Python

Exceptions

Why Programs Crash – and How to Fix Them

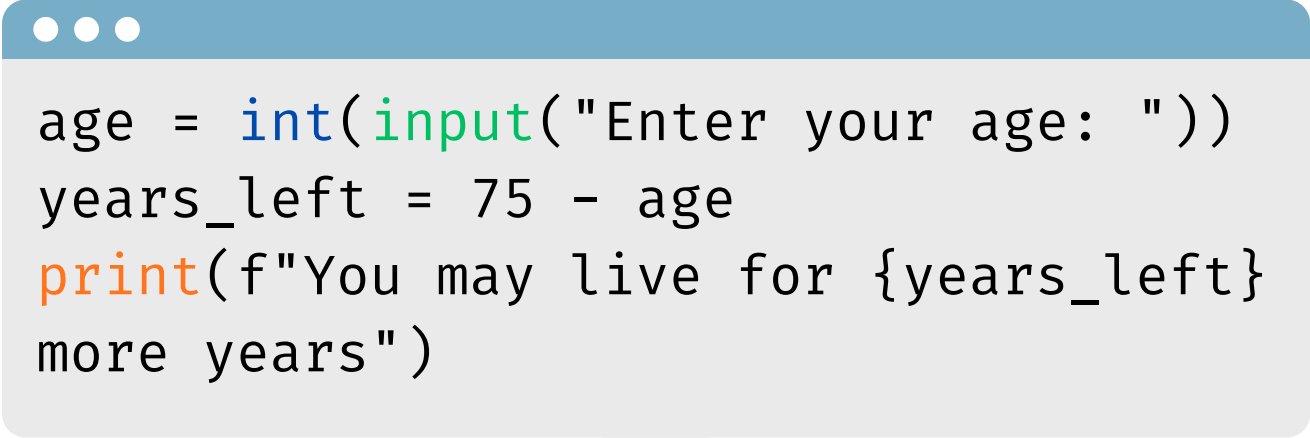
Building smarter programs with Exception Handling

Let's build something interesting...

Problem:

“We want to estimate how many years a person has left based on an average life expectancy (e.g., 75 years).”

Version-1

A code editor window with a blue title bar and three white window control buttons (minimize, maximize, close) on the left. The background is light gray. The code is written in a monospaced font with syntax highlighting: 'int' is blue, 'input' is green, 'print' is orange, and string literals are in black. The code calculates the remaining years to age 75 based on user input.

```
age = int(input("Enter your age: "))  
years_left = 75 - age  
print(f"You may live for {years_left}  
more years")
```

Q: What could go wrong?

Testing

- `abc` → crash
- `-5` → weird result
- `200` → negative years

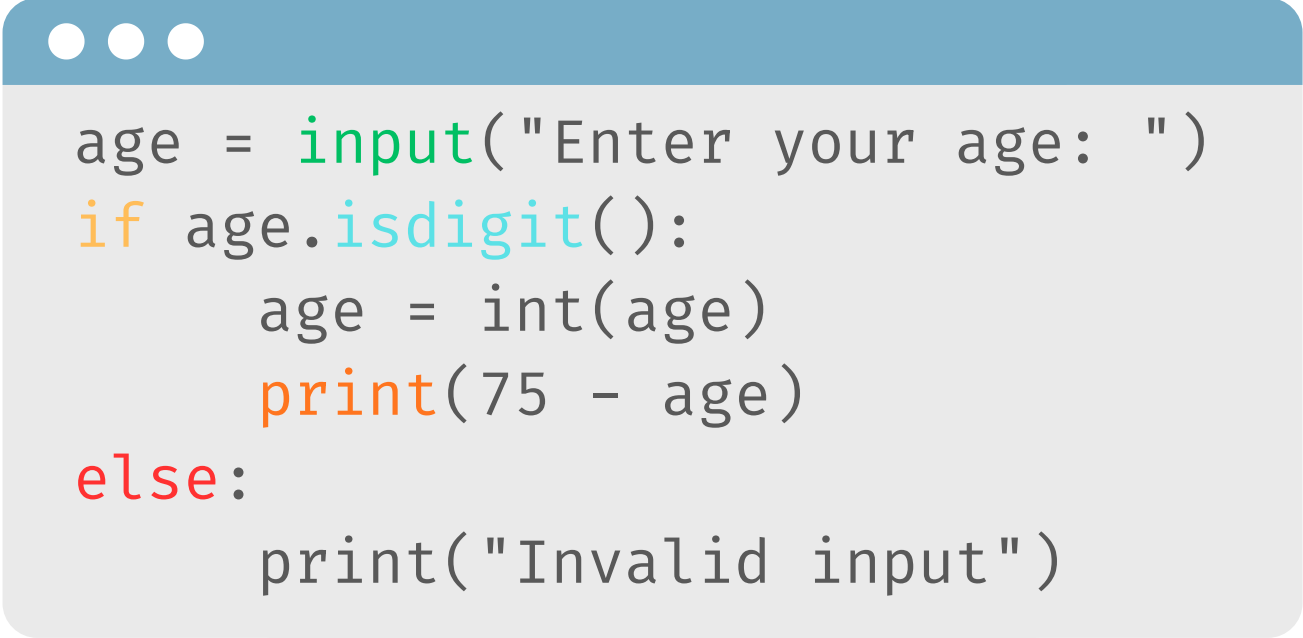
Discussion

- User entered text → program crashed
- User entered unrealistic numbers
- Program trusted user too much

Key message:

- Our program is fragile

Version-2



```
age = input("Enter your age: ")
if age.isdigit():
    age = int(age)
    print(75 - age)
else:
    print("Invalid input")
```

Q: What are the limitations of this version ?

Limitations:

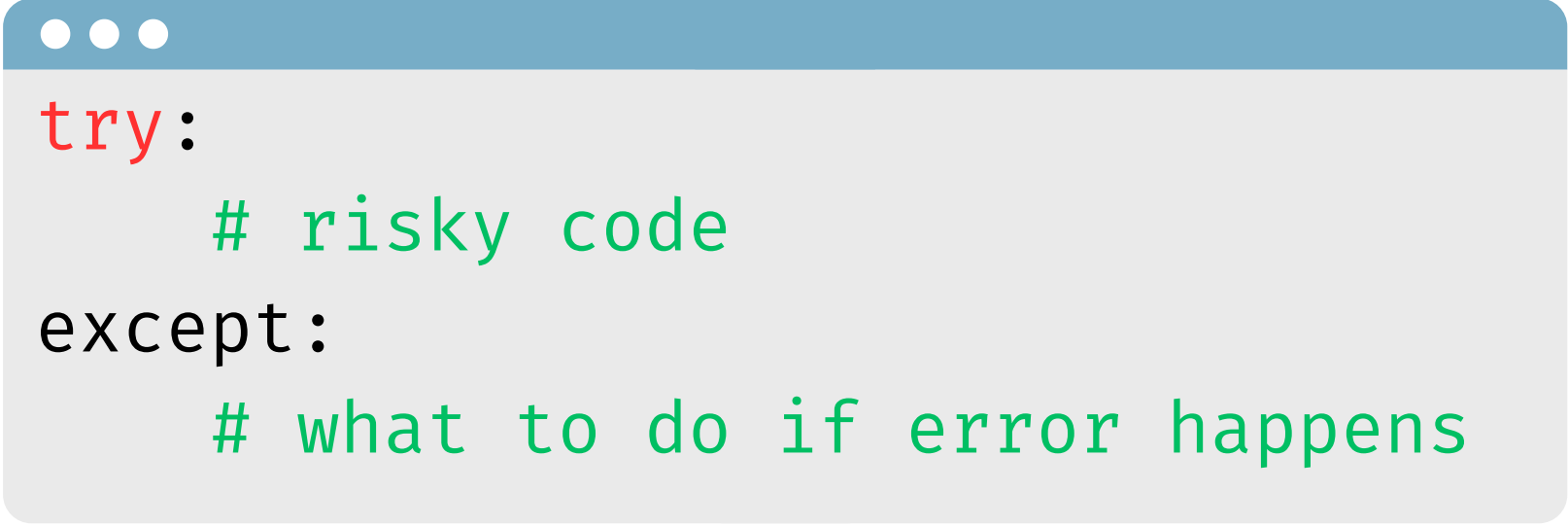
- Doesn't handle negative numbers
- Doesn't handle all edge cases
- Gets complicated quickly

Q: Is there a better way ?

Transition to Pythonic Way

“Instead of checking everything like before...
Python encourages us to try first, then handle errors”

try and except Syntax



```
try:
    # risky code
except:
    # what to do if error happens
```

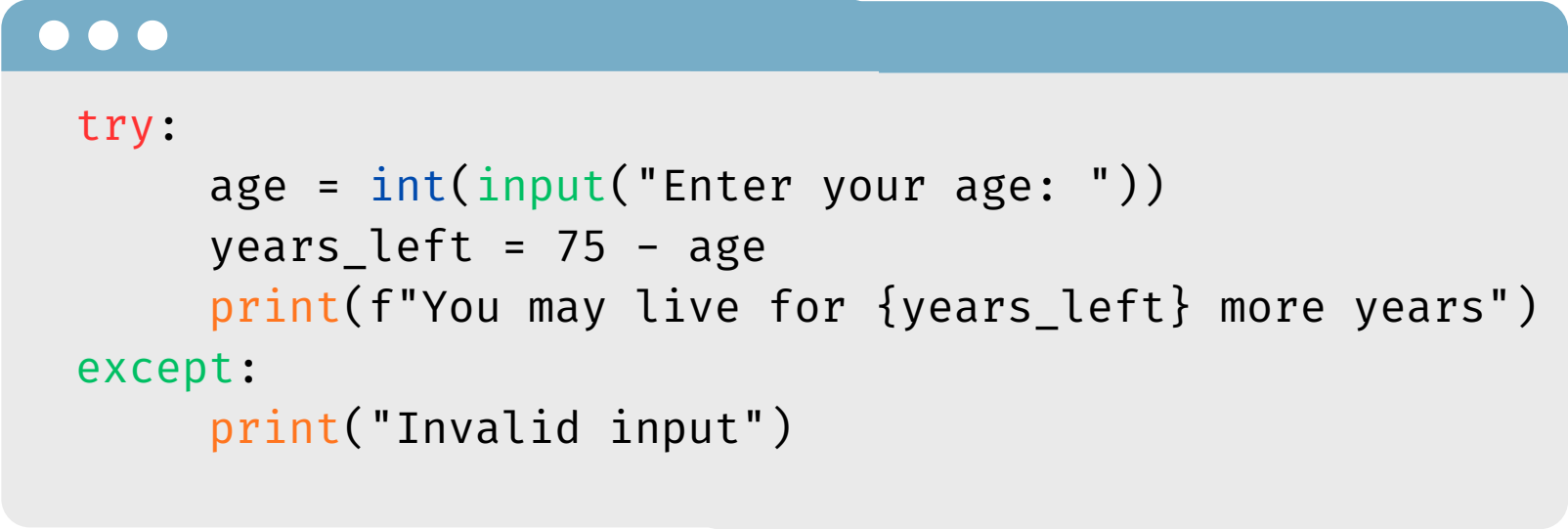
Formal Definition

- try → test a block of code
- except → handle errors if they occur\

Simple explanation:

- “Try it... if it fails, don't crash – recover”

Version-3



```
try:
    age = int(input("Enter your age: "))
    years_left = 75 - age
    print(f"You may live for {years_left} more years")
except:
    print("Invalid input")
```

So much better !!

Improvment

- No crash
- User gets feedback

Q: Is this perfect ?

Problem

- It hides ALL errors
- Not specific

Version-4

```
try:
    age = int(input("Enter your age: "))
    print(75 - age)
except ValueError:
    print("Please enter a valid number")
```

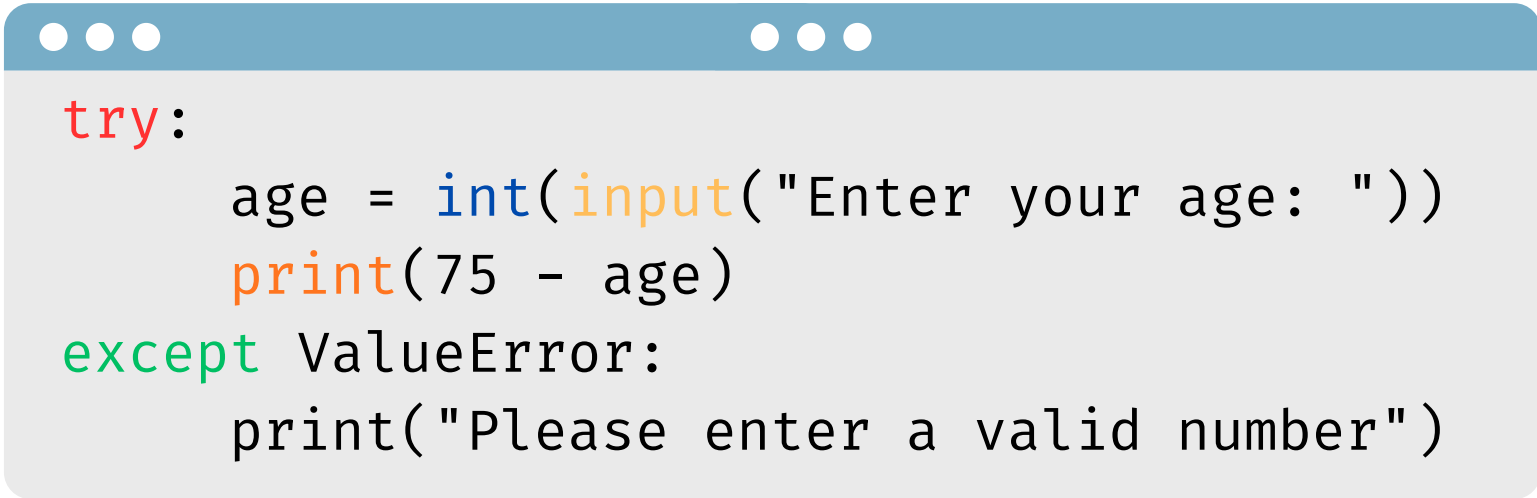
ValueError

- Happens when conversion fails
- Example: "abc" $\rightarrow$ int

More Errors Exist

- `NameError`: You tried to use a variable or function that hasn't been defined yet.
- `TypeError`: You tried to perform an operation on an unsupported data type.
- `AttributeError`: You tried to use a method on an object that doesn't have it.
- `ZeroDivisionError`: The second argument of a division or modulo operation is zero.
- `IndexError`: You tried to access an index in a list that is out of range.

Improvement



```
try:
    age = int(input("Enter your age: "))
    print(75 - age)
except ValueError:
    print("Please enter a valid number")
```

Q: What is wrong with this version?

Subtle Problem (Very Important)

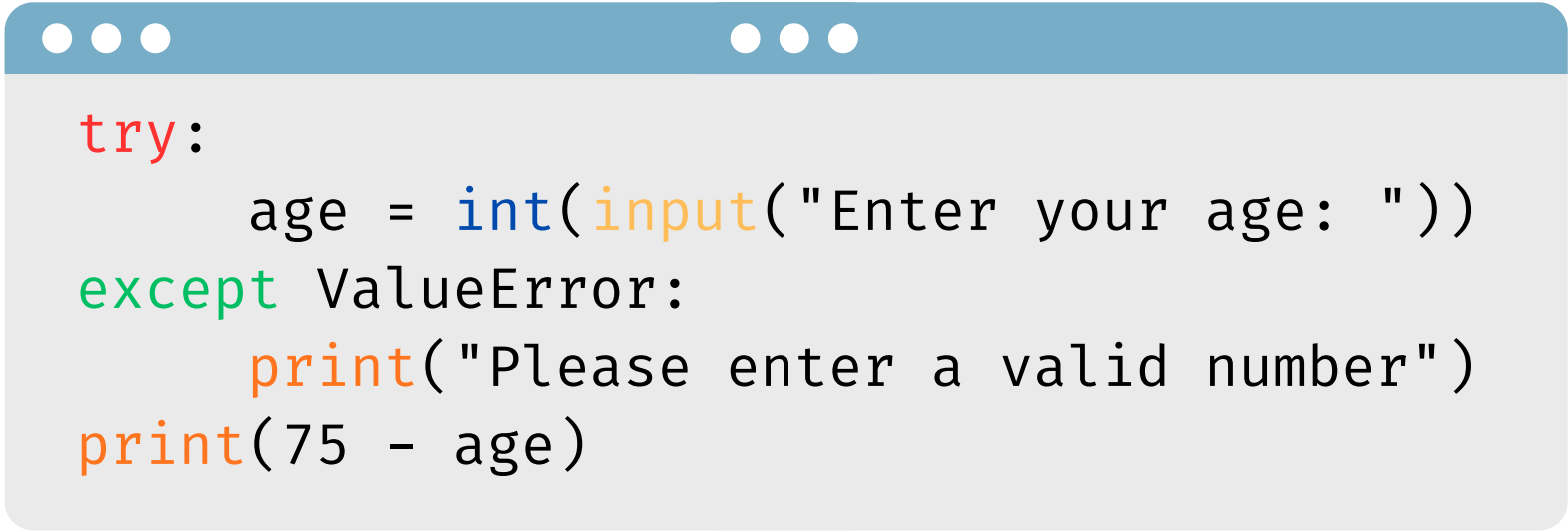
We put too much inside try

- Input → risky
- Printing → NOT risky

Principle:

“Only put risky code inside try”

Version-5



```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a valid number")
print(75 - age)
```

Q: What is wrong ?

Running Version-5

We get:

`NameError / UnboundLocalError`

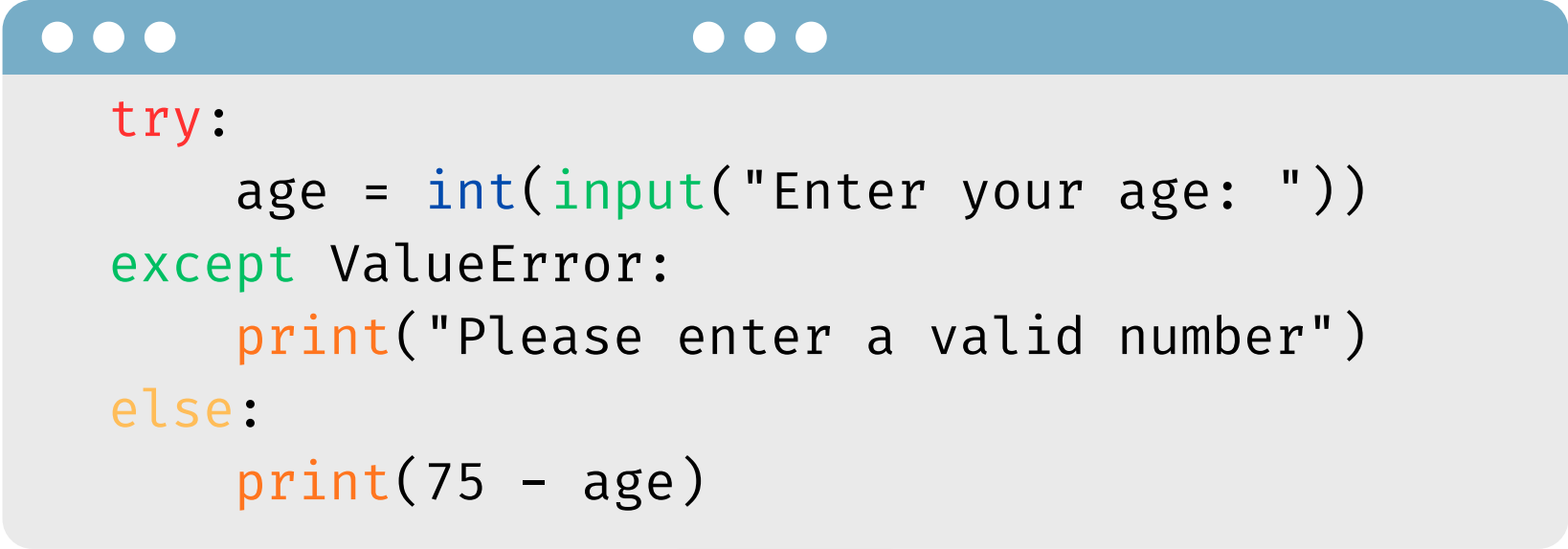
Q: What is wrong ?

Explanation

In Python, the `=` sign is the very last thing to happen. Python evaluates the right side of the equation completely before it "names" the result on the left.

1. Step 1: Run `input()`. User enters "twenty".
2. Step 2: Run `int("twenty")`. BOOM. An exception is raised.
3. The Result: Because the error happened during the conversion, Python immediately jumps to the `except` block. It never reaches the `"="` sign. The variable `age` was never created in the computer's memory. It's a "ghost !"

Version-6



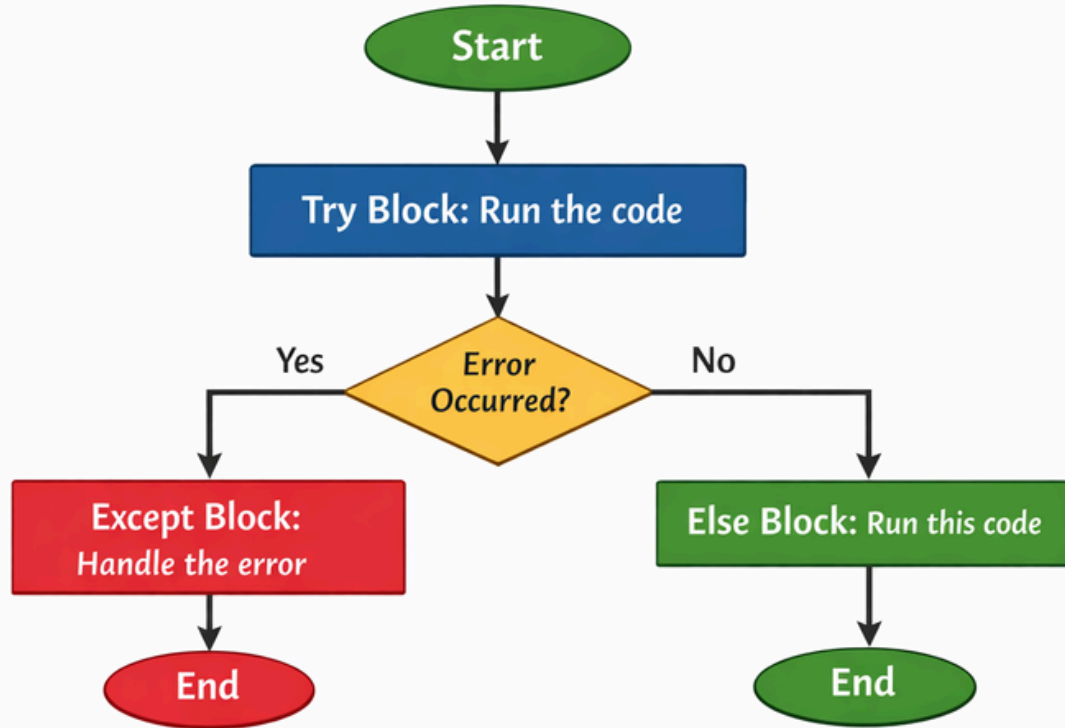
```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a valid number")
else:
    print(75 - age)
```

Q: But how does this work?

Flow Explanation

- Try runs first
- If error $\rightarrow$ go to except
- If NO error $\rightarrow$ go to else

Flow Explanation



Q_1 : What happens after an error?

Q_2 : Should the user get another chance?

Retry Mechanism

```
while True:
    try:
        age = int(input("Enter your age: "))
    except ValueError:
        print("Invalid input. Try again.")
    else:
        print(75 - age)
        break
```

“Now the program doesn’t just survive errors... it recovers and continues”

Can We Create Our Own Errors?

Q: Is a negative age an error for Python?

But....

Is it an error for OUR program?

raise

The raise keyword is a control flow statement that interrupts the normal execution of a program by signaling that an exceptional condition has occurred.

The Syntax:

```
raise ExceptionClass ("optional message")
```

Last-version

```
def get_age():
    while True:
        try:
            age = int(input("Enter your age: "))

            if age < 0 or age > 120:
                raise ValueError("Invalid age range")

            return age

        except ValueError as e:
            print(f"Error: {e}")
```

Introduction to

Programming with Python

Exceptions